

Codificação Para Front-end

05

2026

SENAI - CRTI



Validação e Handoff para Desenvolvimento

Unidade Curricular: Codificação para Front-end

Roteiro da Aula

O que vamos aprender hoje?

01

Situação de Aprendizagem

Contextualização com um cenário real de problema

02

Mobilização de Conhecimentos

Reflexão sobre experiências e conhecimentos prévios

03

Desenvolvimento do Conteúdo

Validação, refinamento, documentação, tokens e handoff

04

Atividades Práticas

Aplicação dos conceitos em exercícios guiados

05

Sistematização e Avaliação

Resumo visual + entregável em grupo

Um projeto que deu errado — e por quê

O Cenário

Uma equipe de desenvolvimento inicia a codificação de um sistema web logo após receber os wireframes do designer. Sem validar o protótipo com usuários, sem documentar os componentes e sem alinhar as especificações visuais, o time começa a implementar as telas.

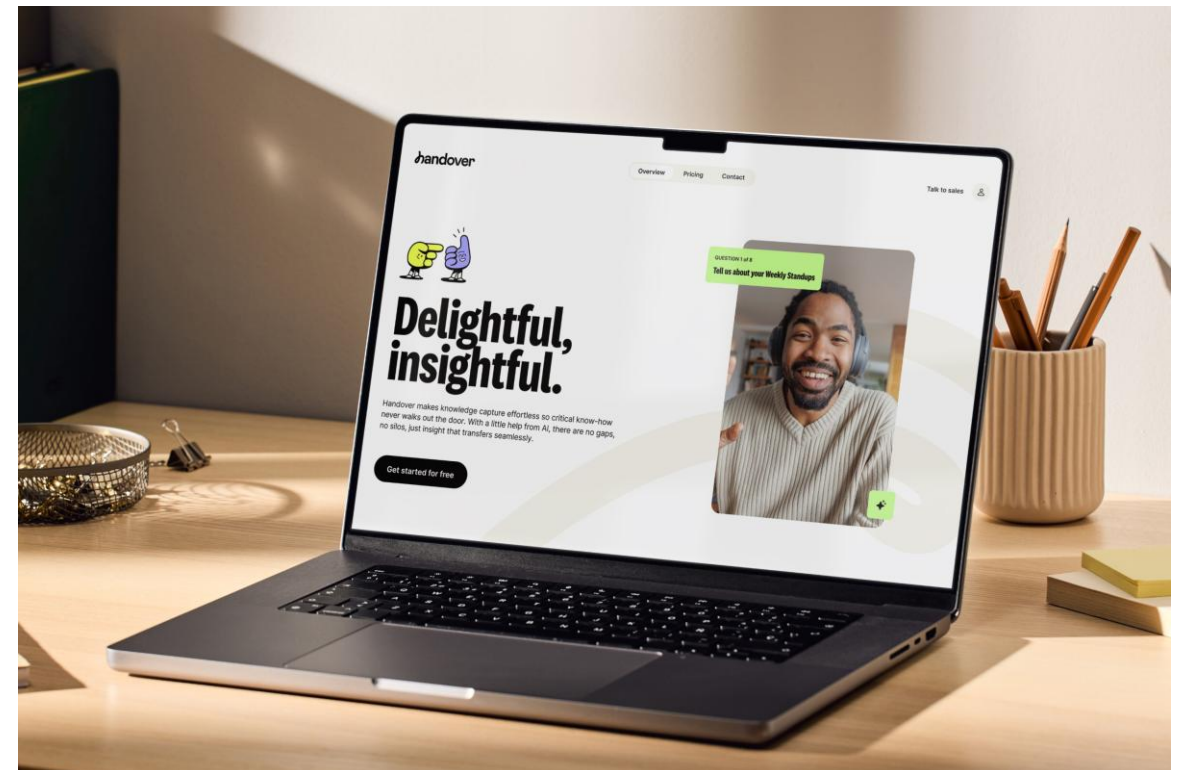
O que aconteceu?

- Botões implementados com cores e tamanhos errados
- Fluxo de navegação quebrado em telas específicas
- Estados de erro e hover nunca especificados
- Retrabalho de 3 semanas para corrigir inconsistências
- Cliente insatisfeito com a entrega final

⊗ **Problema central:** A falta de validação e de comunicação estruturada entre design e desenvolvimento gerou retrabalho, inconsistência visual e falhas de usabilidade.

Esse cenário poderia ser evitado?

Sim — com um processo de validação e handoff bem definido. Antes de escrever qualquer linha de código, design e desenvolvimento precisam falar a mesma língua. Esta aula apresenta as ferramentas e práticas para isso.



O que você já sabe sobre isso?

Antes de avançarmos para o conteúdo novo, vamos refletir juntos. Pense nas experiências que você já teve — como aluno, usuário ou observador de projetos tecnológicos.

“

Questão 1

O que acontece quando um sistema é desenvolvido sem nenhum teste com usuários reais antes da entrega?

”

“

Questão 2

Como garantir que o design criado pelo designer será implementado *exatamente* como planejado pelo desenvolvedor front-end?

”

“

Questão 3

O desenvolvedor sempre entende o que o designer pensou? O que pode falhar nessa comunicação?

”

 Compartilhe suas respostas com a turma. Não há resposta errada — o objetivo é ativar o que você já vivenciou ou observou.

Validação de Protótipos

Validar um protótipo significa testar se a solução proposta realmente funciona para o usuário antes de iniciar o desenvolvimento. É o passo que transforma suposições em certezas.

O que é validação?

Processo de verificar se o protótipo atende às necessidades reais do usuário, com foco em usabilidade, clareza e fluxo de navegação.

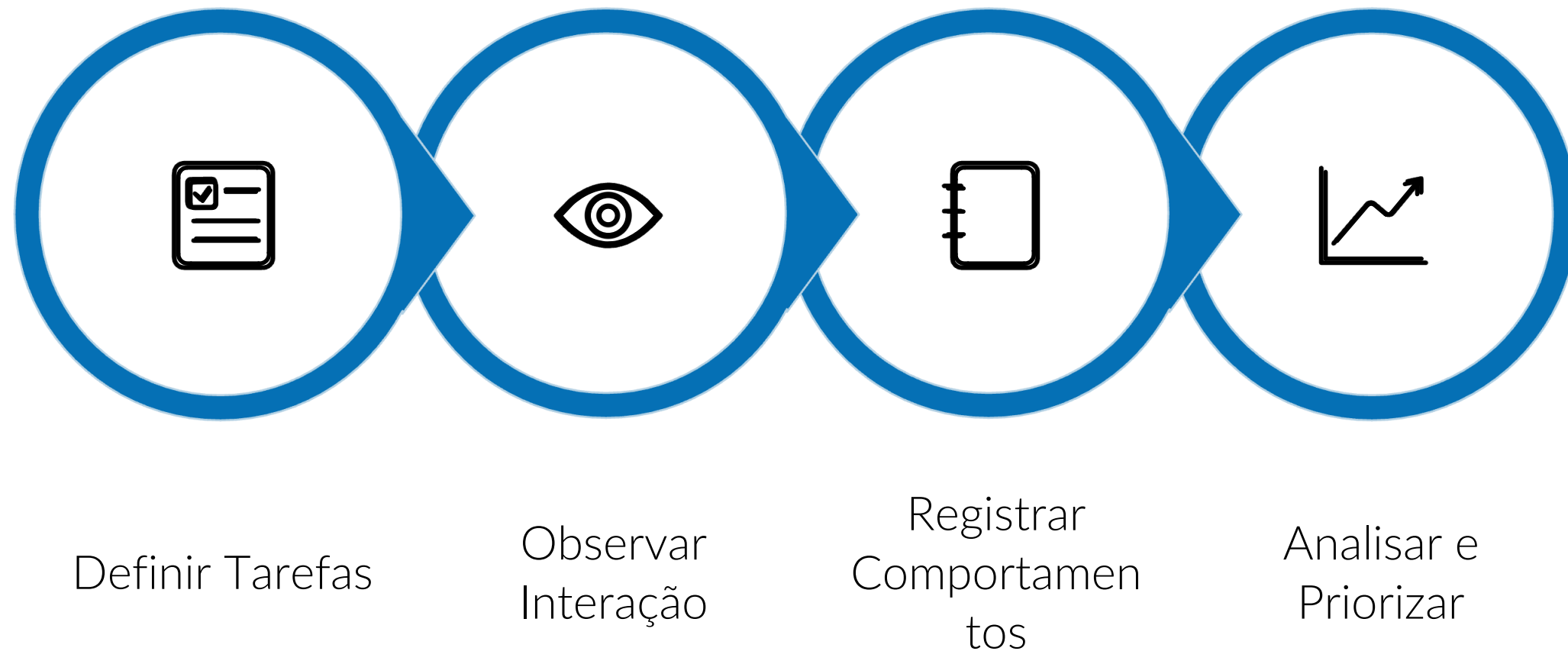
Testes com Usuários

Sessões onde usuários reais interagem com o protótipo enquanto os observadores registram dificuldades, dúvidas e comportamentos inesperados.

Identificação de Problemas

Análise dos registros para mapear erros recorrentes, pontos de confusão, elementos ignorados e inconsistências no fluxo.

Como conduzir um teste de usabilidade?



Um bom teste de usabilidade não precisa ser complexo. Até 5 usuários são suficientes para identificar cerca de 85% dos problemas de uma interface, segundo Nielsen Norman Group.



O que observar durante o teste?

Hesitação e pausas longas

Quando o usuário para e olha para a tela sem saber o que fazer, o fluxo ou a hierarquia visual está com problema.

Erros de clique

Cliques em elementos que não são interativos, ou ausência de clique em elementos que deveriam ser óbvios.

Verbalização de confusão

Frases como "não entendi o que esse botão faz" ou "não sei onde estou no sistema" sinalizam falhas de feedback ou navegação.

Comportamento inesperado

Quando o usuário tenta resolver a tarefa de uma forma que o designer não previu — isso revela lacunas no modelo mental do design.

Refinamento com Base em Feedback

O que é refinamento?

Refinamento é o processo de ajustar o protótipo com base nos problemas identificados nos testes. Não é redesign completo — é correção cirúrgica e iterativa, priorizando o impacto para o usuário.

Critérios de priorização

- **Alta prioridade:** bloqueia o usuário de completar a tarefa
- **Média prioridade:** gera confusão mas não impede o uso
- **Baixa prioridade:** melhoria estética ou de conforto

Ciclo de Refinamento

Testar → Registrar → Priorizar → Ajustar → Testar novamente

Cada ciclo aproxima o protótipo de uma experiência mais consistente, funcional e pronta para desenvolvimento.

Antes e Depois: Exemplo de Refinamento

ANTES DO REFINAMENTO



DEPOIS DO REFINAMENTO



Pequenos ajustes fazem grande diferença. O refinamento não é opcional — é parte essencial do processo de design profissional.

Revisando o Protótipo: Identificando Problemas

 **Duração estimada:** 20 minutos | **Formato:** Individual ou dupla

Utilize o protótipo desenvolvido nas aulas anteriores e realize uma análise crítica de usabilidade. O objetivo é identificar problemas antes de chegar ao código.

1

Percorra o protótipo

Navegue por todas as telas como se fosse um usuário real tentando completar as principais tarefas do sistema.

2

Registre problemas

Para cada problema encontrado, anote: qual tela, qual elemento, qual dificuldade e qual impacto para o usuário.

3

Priorize e ajuste

Classifique os problemas (alta/média/baixa prioridade) e realize os ajustes necessários diretamente no protótipo.



Design Specification Product Requirements Document

I. Introduction

A. Product Overview

The **AireMix Task Manager** is a comprehensive project management tool designed to streamline task allocation and progress tracking for teams of all sizes. It provides an intuitive interface for creating, assigning, and monitoring tasks, ultimately enhancing team collaboration and productivity.

B. Objectives

The primary objectives of the **AireMix Task Manager** are:

- To simplify task management processes for teams.
- To improve communication and collaboration among team members.
- To increase overall productivity and efficiency.

C. Scope

The scope of the **AireMix Task Manager** includes:

- Task creation and assignment.
- Task tracking and status updates.
- User authentication and access control.
- Reporting and analytics features.

II. Stakeholders

A. Stakeholder List

- [Your Company Name] Management Team** - Responsible for project oversight and decision-making.
- Development Team** - Responsible for implementing the product requirements.
- Marketing Team** - Responsible for promoting the product to potential users.
- End Users** - Individuals who will use the **AireMix Task Manager** to manage their tasks.

B. Stakeholder Requirements

DESENVOLVIMENTO · BLOCO 3

Documentação UI/UX

Documentar uma interface é traduzir as decisões de design em linguagem técnica que o desenvolvedor front-end consiga implementar com precisão e autonomia. Uma boa documentação reduz ambiguidade, elimina perguntas desnecessárias e garante consistência.

O que deve estar na documentação UI?



Especificação de Telas

Nome da tela, objetivo, contexto de acesso e descrição dos elementos visuais com medidas, cores, fontes e espaçamentos.



Componentes

Descrição de cada componente (botões, inputs, cards, modais) com suas variações: tamanhos, estilos e propriedades.



Estados de Interação

Como cada elemento se comporta nos estados: **default**, **hover**, **active**, **focus**, **disabled** e de **erro**.



Responsividade

Comportamento dos elementos nos breakpoints principais (mobile, tablet, desktop), com especificações de layout para cada.

Exemplo de Especificação de Componente

Botão Primário — Especificação

Propriedade	Valor
Background (default)	#0570b5
Background (hover)	#02294d
Cor do texto	#FFFFFF
Border radius	4px
Padding	12px 24px
Font size	16px / 1rem
Font weight	600 (SemiBold)
Estado disabled	Opacidade 40%, cursor not-allowed

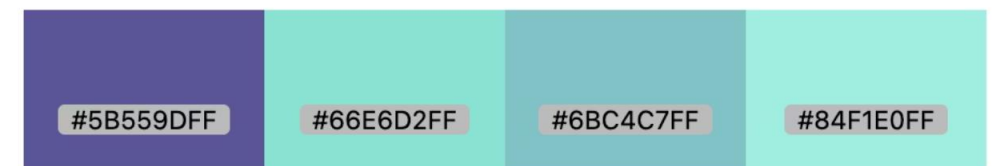
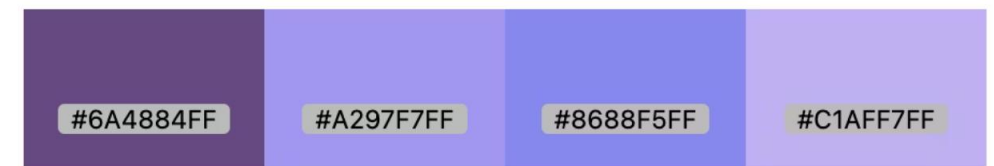
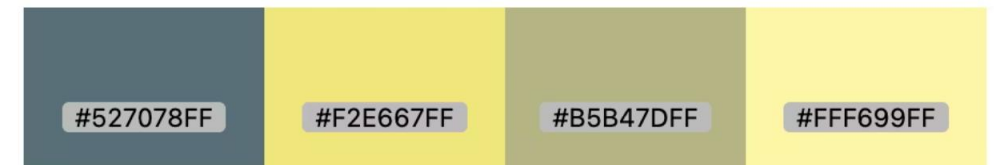
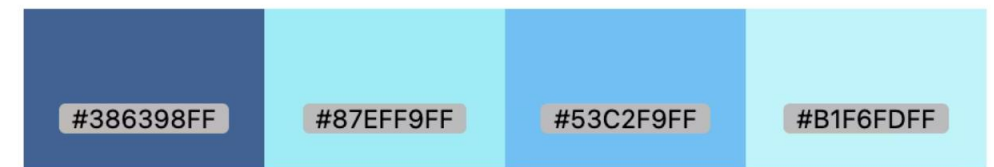
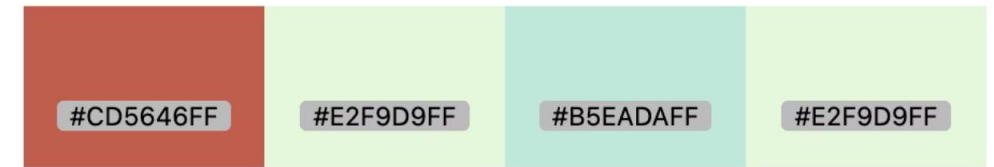
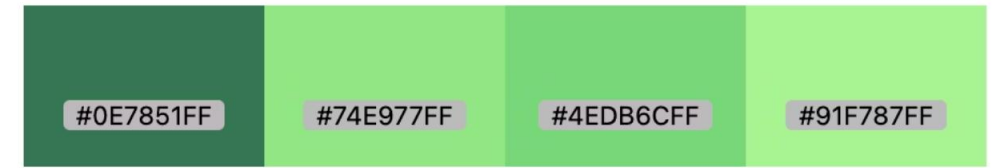
Por que especificar estados?

Estados como `hover`, `focus` e `disabled` são frequentemente esquecidos no design, mas são essenciais para acessibilidade e para uma experiência de usuário completa.

 Sem especificação de estados, o desenvolvedor define seu próprio padrão — gerando inconsistência visual em todo o sistema.

Design Tokens: A Linguagem Comum entre Design e Código

Design Tokens são variáveis nomeadas que armazenam os valores fundamentais do sistema de design — cores, tipografia, espaçamentos, bordas e sombras. Eles criam uma única fonte da verdade compartilhada entre designers e desenvolvedores.



Categorias de Design Tokens



Cores

Paleta primária, secundária, estados (sucesso, erro, alerta) e tons neutros. Ex: `--color-primary: #0570b5`



Tipografia

Família da fonte, tamanhos de escala, pesos e altura de linha. Ex: `--font-size-md: 1rem`, `--font-weight-bold: 700`



Espaçamento

Escala de margins e paddings baseada em múltiplos. Ex: `--spacing-sm: 8px`, `--spacing-lg: 24px`



Bordas e Sombras

Raios de borda e elevações de sombra padronizados. Ex: `--border-radius-md: 4px`, `--shadow-card: 0 2px 8px rgba(0,0,0,0.1)`

Design Tokens em CSS e Angular

Definição em CSS (variáveis globais)

```
:root {
  --color-primary: #0570b5;
  --color-secondary: #51c8e6;
  --color-dark: #02294d;

  --font-family-base: 'Inter', sans-serif;
  --font-size-base: 1rem;
  --font-size-lg: 1.25rem;

  --spacing-xs: 4px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;

  --border-radius-sm: 2px;
  --border-radius-md: 4px;
}
```

Uso em componente Angular (SCSS)

```
.btn-primary {
  background-color: var(--color-primary);
  color: #fff;
  padding: var(--spacing-sm) var(--spacing-md);
  border-radius: var(--border-radius-md);
  font-family: var(--font-family-base);
  font-size: var(--font-size-base);

  &:hover {
    background-color: var(--color-dark);
  }
}
```

✔ Com tokens, mudar a cor primária do sistema inteiro exige alterar **apenas uma linha** de código.

Por que usar Design Tokens em Angular?

1

Consistência global

Todos os componentes usam os mesmos valores de cor, fonte e espaçamento, garantindo identidade visual uniforme.

2

Manutenção simplificada

Alterações no sistema de design exigem mudança em um único lugar — os tokens — sem necessidade de refatorar todos os componentes.

3

Colaboração design-dev

Tokens nomeados criam vocabulário comum. Quando o designer fala "cor primária", o desenvolvedor sabe exatamente qual variável utilizar.

4

Suporte a temas

Com tokens, implementar modo escuro ou personalização por cliente (theming) torna-se uma tarefa direta e controlada.

Definindo Tokens de Design do Seu Projeto

 **Duração estimada:** 15 minutos | **Formato:** Individual

Com base no protótipo do seu projeto, crie um arquivo de tokens de design. Use as categorias abaixo como guia.



Cores

Defina: primária, secundária, fundo, texto, estados de sucesso, erro e alerta.



Tipografia

Defina: família da fonte, tamanhos (base, heading, small), pesos utilizados.



Espaçamentos

Defina uma escala de espaçamento (xs, sm, md, lg, xl) usando múltiplos de 4px ou 8px.

 Entregável: arquivo `.css` com as variáveis definidas no seletor `:root`.

DESENVOLVIMENTO · BLOCO 5

Handoff: A Entrega do Design para Desenvolvimento

handoff é o processo formal de transferência do design para a equipe de desenvolvimento. Não é apenas enviar arquivos — é garantir que toda a informação necessária esteja organizada, acessível e compreensível para quem vai codificar.



O que compõe um handoff completo?



Um handoff bem estruturado é o principal fator para que o desenvolvimento seja feito corretamente na primeira tentativa.

Boas Práticas de handoff

✓ Faça desta forma

- Nomeie todas as camadas e componentes no Figma/XD
- Organize as telas por fluxo ou seção do sistema
- Especifique todos os estados de cada componente
- Inclua anotações diretamente nas telas
- Realize reunião de alinhamento com o time de dev
- Documente as decisões de design e o porquê delas

✗ Evite

- Enviar arquivos sem estrutura ou organização
- Assumir que o desenvolvedor "vai entender" sem documentação
- Não especificar estados interativos (hover, error, disabled)
- Usar medidas inconsistentes sem sistema definido
- Não revisar o protótipo antes de entregar
- Desaparecer após o envio dos arquivos

Revisão de Fluxo de Navegação

Antes do handoff, todo o fluxo de navegação do sistema deve ser revisado com olhar técnico — verificando se cada tela está conectada corretamente, se os caminhos fazem sentido para o usuário e se os desenvolvedores têm clareza sobre cada transição.

1

Mapear Telas

Listar todas as telas do sistema com seus nomes e identificadores únicos

2

Conectar Fluxos

Indicar qual ação leva de uma tela para outra e qual o tipo de transição

3

Verificar Caminhos

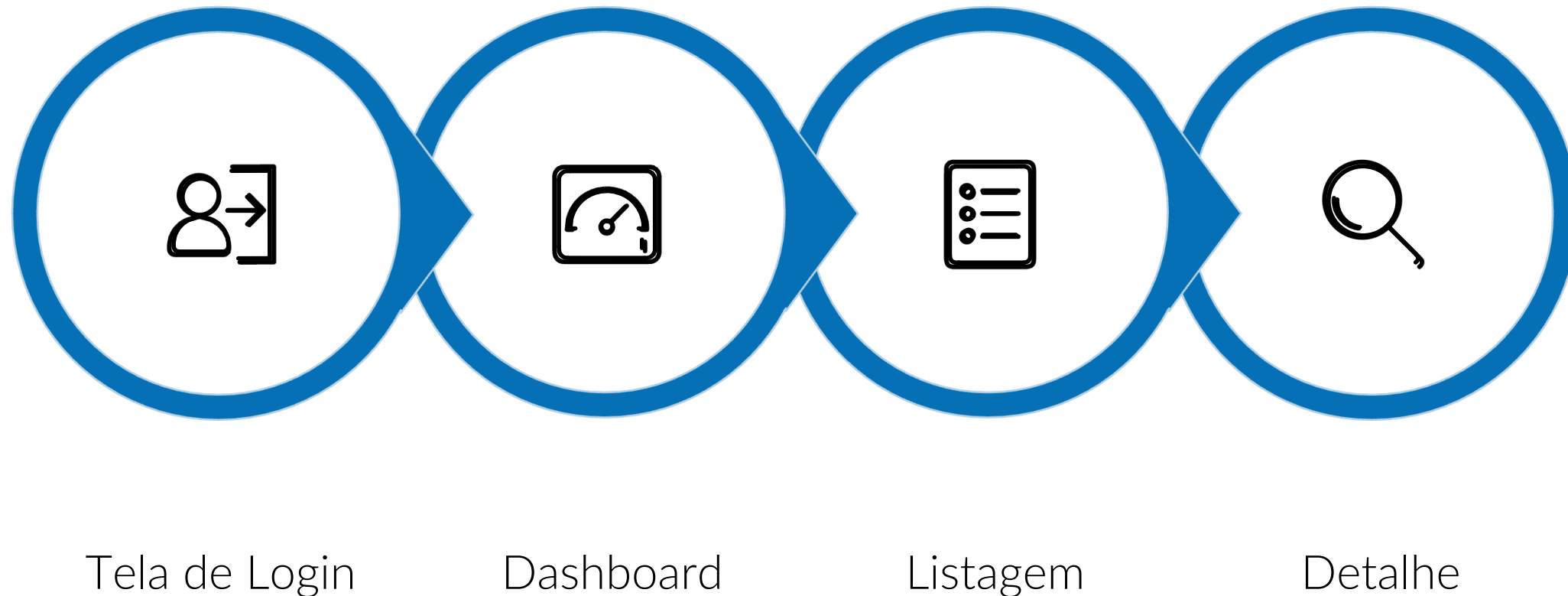
Confirmar que todos os caminhos principais e alternativos (erros, estados vazios) estão cobertos

4

Validar com Dev

Alinhar com o desenvolvedor se o fluxo é tecnicamente viável para implementação em Angular

Exemplo de Fluxo de Navegação



O mapeamento do fluxo deve cobrir tanto os caminhos ideais (happy path) quanto os caminhos alternativos, como erros de autenticação, estados vazios e mensagens de validação de formulário.

Mapeando o Fluxo de Navegação do Seu Projeto

 **Duração estimada:** 15 minutos | **Formato:** Dupla ou trio

Utilize papel, quadro branco ou ferramenta digital para mapear o fluxo completo de navegação do protótipo do seu projeto.

1 Liste todas as telas do sistema

Crie uma lista com o nome e a função de cada tela (ex: Login, Cadastro, Dashboard, Listagem, Detalhe, Configurações).

2 Conecte as telas com ações

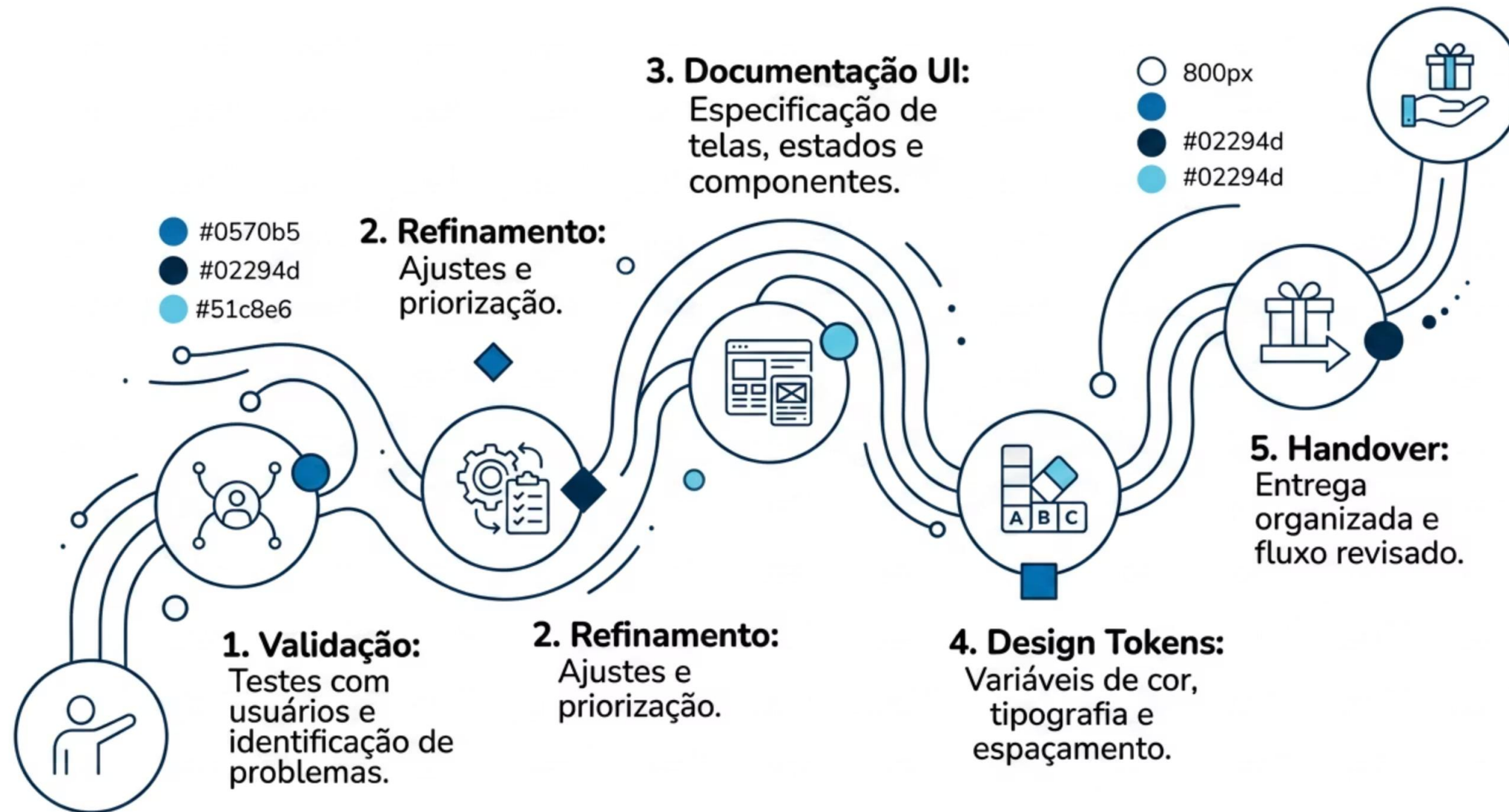
Para cada tela, indique: qual ação ou evento leva o usuário para outra tela e qual é a tela de destino.

3 Inclua caminhos alternativos

Adicione ao mapa os estados de erro, telas de confirmação, modais e qualquer desvio do fluxo principal.

 **Entregável:** diagrama de fluxo de navegação (pode ser feito no Figma, Miro, draw.io ou desenhado à mão).

Resumo Visual: Do Protótipo ao Código



Os 5 Pilares de um handoff de Qualidade

Validação

Protótipo testado com usuários reais e problemas documentados antes do desenvolvimento

Refinamento

Ajustes baseados em feedback com critérios claros de prioridade e iteração

Documentação

Telas, componentes e estados especificados de forma clara e técnica

Design Tokens

Variáveis de design mapeadas para variáveis CSS/SCSS prontas para uso em Angular

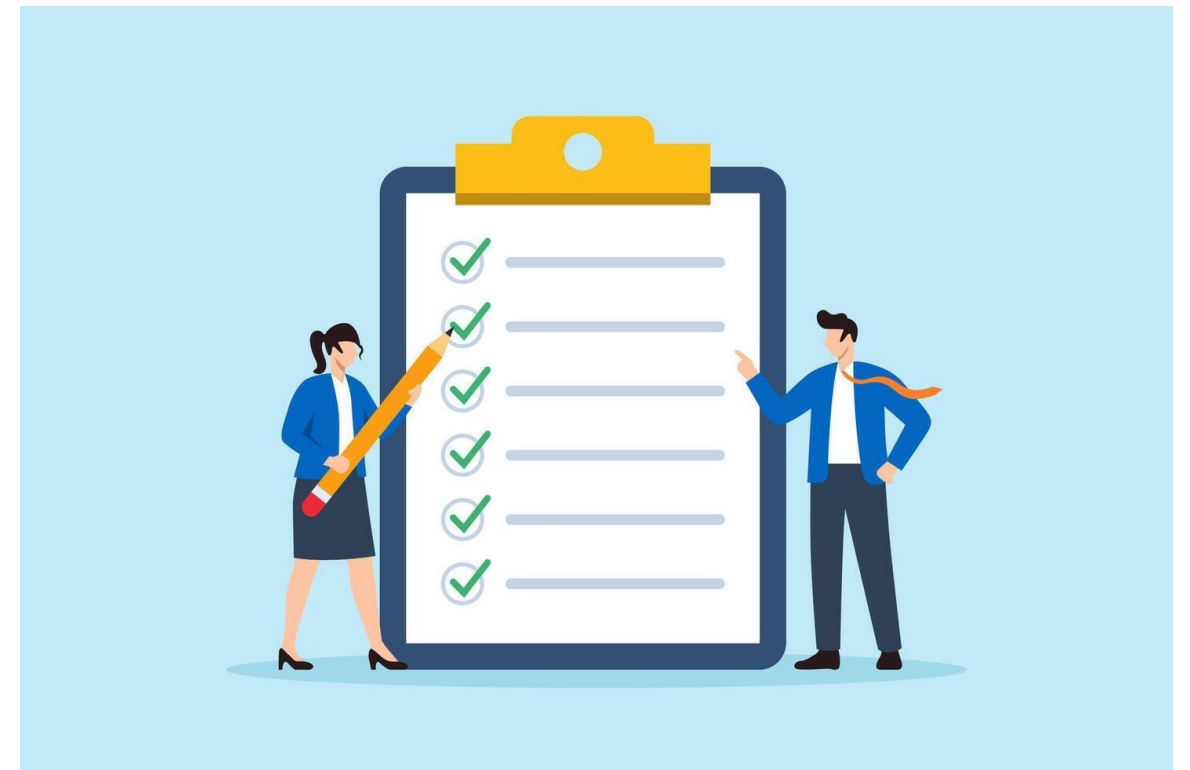
Fluxo de Navegação

Mapa completo de todas as telas, transições e caminhos alternativos do sistema

ATIVIDADE FINAL · 30 MINUTOS

Preparando o Projeto para Desenvolvimento

Chegou a hora de integrar tudo o que aprendemos! Em grupos de 3 a 4 pessoas, vocês vão preparar o handoff completo de um protótipo para o time de desenvolvimento front-end.



Atividade Final: Tarefas do Grupo

1

Revisar e ajustar o protótipo

Aplicar os refinamentos identificados nos testes de usabilidade. Garantir que todos os estados interativos estão representados.

2

Criar documentação básica de UI

Para ao menos 3 telas do protótipo: descrição da tela, lista de componentes utilizados e especificação de estados.

3

Definir Design Tokens

Criar arquivo CSS com variáveis de cor (mínimo 5 tokens) e tipografia (mínimo 3 tokens) do sistema de design do projeto.

4

Mapear fluxo completo

Diagrama com todas as telas, ações de navegação e caminhos alternativos (erros, confirmações, estados vazios).

5

Apresentar para a turma

Apresentação de 5 minutos simulando um handoff real: "Como desenvolvedor, isso é tudo que você precisa saber."

Competências Desenvolvidas Nesta Aula



Validar protótipos

Conduzir testes com usuários, registrar observações e identificar problemas de usabilidade com critério técnico.



Refinar soluções de UI/UX

Priorizar e aplicar melhorias baseadas em feedback real, usando critérios de impacto e esforço.



Documentar interfaces

Produzir documentação técnica de telas e componentes que seja útil e clara para equipes de desenvolvimento.



Trabalhar com Design Tokens

Definir e implementar variáveis de design em CSS e aplicá-las em projetos Angular com consistência.



Realizar handoff

Preparar e entregar o pacote completo de design para desenvolvimento com organização e comunicação eficaz.



Design bem feito é código bem escrito

A fronteira entre design e desenvolvimento deixa de ser um obstáculo quando a comunicação é estruturada, os padrões são definidos e o protótipo foi validado antes de chegar ao código.

O processo que aprendemos hoje — validar, refinar, documentar, tokenizar e entregar — é o que separa projetos de qualidade profissional de projetos cheios de retrabalho.

